TPN°1 : Introduction au traitement images

Une image numérique peut être représentée par une matrice dans laquelle chaque case représente un pixel, unité minimale adressable par le contrôleur vidéo et supposé de couleur uniforme. Le format de l'image (png, jpeg, etc) désigne la manière dont cette matrice est représentée en mémoire.

- Pour une image en noir et blanc, chaque pixel ne peut prendre que deux valeurs, ce qui permet de l'encoder sur un seul bit ;
- Pour une image en niveau de gris, chaque teinte de gris (en passant du noir au blanc) est encodée sur un octet, ce qui permet de représenter 28 = 256 nuances de gris ;
- Pour une image en couleur, chaque pixel et encodé sur trois octets, chaque octet représentant une nuance de couleur. Par exemple, dans le cas du format RGB (Red, Green, Blue), le premier octet dose la quantité de rouge, le second la quantité de vert et le troisième la quantité de bleu pour une synthèse additive ;
- Enfin, les pixels d'une image en couleur peuvent être encodés par quatre octets au format RGBA (A pour alpha), le quatrième octet codant la transparence du pixel.

1. Manipulation d'image en Python :

- La fonction plt.imread(source) permet de convertir une image au format courant (png, jpeg, etc) en un tableau numpy. Elle prend en argument une chaîne de caractère décrivant le chemin d'un fichier présent sur votre ordinateur ou l'adresse http d'une image présente sur le net.
- La fonction plt.imshow(tab) permet de visualiser l'image décrite par un tableau. À noter, pour visualiser une image en niveau de gris il faut préciser l'échelle chromatique à utiliser, ici :

```
import numpy as np
import matplotlib.pyplot as plt
```

plt.imshow(tab, cmap='gray').

- Le script qui suit doit vous permettre de charger une image qui servira de modèle puis la visualiser:

```
image = plt.imread("test1.jpg")
plt.imshow(image)
plt.show()
```

- Taille d'une image : Ainsi définie im est tableau numpy dont les éléments sont de type np.uint8 (entiers non signés représentés sur 8 bits, autrement dit un entier compris entre 0 et 255). La méthode shape donne les dimensions de ce tableau : im.shape = (n, p) pour une image en niveau de gris, im.shape = (n, p, 3) pour une image couleur, im.shape = (n, p, 4) pour une image avec composante alpha. L'entier n est le nombre de lignes de l'image et p le nombre de colonnes, le pixel de coordonnées (0,0) est situé en haut à gauche de l'image.

```
image.shape
```

300, 300, 3)

2. Fonctions élémentaires :

Question 1. Rédiger une fonction symetrie qui prend en argument une image et retourne une nouvelle image, obtenue par symétrie autour d'un axe vertical passant par le milieu de l'image. Indication : la fonction np.zeros_like(m) crée un tableau nul de même type et de mêmes dimensions que le tableau m

```
def symetrie(im):
    n, p, c = im.shape
    im_sym = np.zeros_like(im)
    for i in range(n):
        for j in range(p):
            im_sym[i, j] = ....
    return im_sym
```

```
plt.imshow(symetrie(image))
plt.show()
```

Question 2. Rédiger une fonction negatif qui prend en argument une image et retourne son négatif, c'est-à-dire l'image dans laquelle chaque composante de couleur de chaque pixel est remplacée par sa valeur complémentaire dans l'intervalle [0,255].

```
def negatif(im):
    n, p, c = im.shape
    im_neg = np.zeros_like(im)
    for i in range(n):
        for j in range(p):
```

```
plt.imshow(negatif(image))
plt.show()
```

Conversion en niveau de gris : Pour convertir une image couleur en niveau de gris, un pixel représenté par ses composantes (r,g,b) doit être remplacé par un pixel à une seule composante y ∈ [0,255] appelée luminance. Cette quantité, qui traduit la sensation visuelle de luminosité, dépend de manière inégale des trois composantes RGB. La formule communément recommandée pour cette conversion est la suivante : y = 0,2126 r + 0,7152 g + 0,0722 b (y étant arrondi à l'entier le plus proche) (pour un œil humain le vert parait plus lumineux que le rouge, lui-même plus lumineux que le bleu).

Question 4. Rédiger une fonction niveau de gris qui prend an argument une image couleur et retourne une nouvelle image convertie en niveau de gris.

```
def niveaudegris(im):
    n, p, c = im.shape
    im_lum = np.zeros((n, p), dtype=np.uint8)
    for i in range(n):
        for j in range(p):
            im_lum[i, j] = np.round(.....)
    return im_lum
```

```
plt.imshow(niveaudegris(image), cmap='gray')
plt.show()
```

Question 5. Ecrire une fonction qui renvoie le canal rouge de l'image.

```
def image_rouge(im):
    photo_rouge=np.zeros(im.shape, dtype=int)  # créer une matrice de même dimension
    photo_rouge[:,:,0] = ....
    return photo_rouge
```

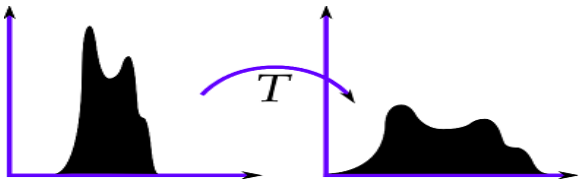
```
plt.imshow(image_rouge(image))
plt.show()
```

Question 6. Ecrire une fonction qui renvoie le canal vert de l'image.
Question 7. Ecrire une fonction qui renvoie le canal bleu de l'image.

3. Egalisation de l'hitogramme d'une image en niveau de gris

L'histogramme d'une photo permet de compter le nombre de pixels d'un niveau de gris donné. En traitement d'images, l'égalisation d'histogramme est une méthode d'ajustement du contraste d'une image numérique qui utilise l'histogramme. Elle consiste à appliquer une transformation T sur chaque pixel de l'image, et donc d'obtenir une nouvelle image à partir d'une opération indépendante sur chacun des pixels. Cette transformation est construite à partir de l'histogramme cumulé de l'image de départ.

Question 8. Écrire une fonction histo(im), qui prend en argument une image en niveau de gris (décrite par une matrice dont chaque pixel est représenté seulement par le niveau de gris). Cette fonction doit



renvoyer une liste de taille 256 : en première position (indice 0), le nombre de pixels noirs (gris 0), en deuxième position (indice 1), le nombre de pixels gris 1, . . . , en dernière position (255), le nombre de pixels blancs (gris 255).

```
def histo(im):
    H = np.zeros(256)
    for i in range(len(im)):
        for j in range(len(im[0])):
            ....
    return H
```

```
plt.plot(np.arange(256),histo(niveaudegris(image))) #X et Y sont les listes (ou tabl
plt.show() #pour afficher
```

```
plt.plot(np.arange(256),histo(image2)) #X et Y sont les listes (ou tableaux numpy) d
plt.show() #pour afficher
```

```
image2 = io.imread("test2.jpg")
image2 = niveaudegris(image2)
plt.imshow(image2, cmap='gray')
plt.show()
```

Question 9. Écrire une fonction egaliser_histogramme(H, gray_image), qui prend en paramètre un histogramme H et une image en niveaux de grus gray_image, la fonction retourne l'histogramme égalisé.

```
def egaliser_histogramme(H, gray_image):
    # histogramme cumulé
    hc = np.zeros(len(H))
    hc[0] = H[0]
    for i in range(1, len(H)):
        ....
    # normaliser lhistogramme cumulé
    he = (hc/gray_image.size)*255
    return he
```

Question 10. Écrire une fonction augmenter_contraste(gray_image), qui prend en paramètreune image en niveaux de grus gray_image, la fonction un nouvelle image avec un constraste augumenté.

```
def augmenter_contraste(gray_image):
    H = histo(gray_image)
    he = egaliser_histogramme(H, gray_image)
    # Utilise he comme table de conversion des niveaux de gris
    new_image = gray_image.copy()
    for i in range(0,gray_image.shape[0]):         # énumère les lignes
        for j in range(0,gray_image.shape[1]):     # énumère les colonnes
            new_image[i,j] = .....
    return new_image
```

```
plt.imshow(augmenter_contraste(image2), cmap='gray')
plt.show()
```